

Lab 3 — Architectural Justification

Student Club Event Ticketing & Budget Portal (SCETBP) · Problem Statement #10
Aarush Muralidhara · PES1UG24CS010 · Class 5A

Architecture Selection: We chose the **Microservices Architecture** for the Student Club Event Ticketing & Budget Portal system.

Reason 1 — The system carries two workloads of opposite shape. Budget approval is human-paced and low-volume: a proposal moves Faculty Coordinator → Finance Office → Dean over days, with a handful of writes per event. QR check-in is the opposite — hundreds of scans compressed into the fifteen minutes before an event starts. Under a Layered or Client-Server design both workloads share one deployment, one thread pool and one database connection pool, so a check-in rush at a large fest directly degrades the approval portal for everyone else. Splitting them lets the Check-In Validation Service run at many replicas for the duration of an event while the Event Proposal Manager and Budget Approval Service stay at one, which is the only sizing that matches how this system is actually used.

Reason 2 — Independent release cycles and fault isolation. Ticketing rules are volatile: QR format, reissue and revoke behaviour change from one semester and one club to the next. The approval chain is the opposite — it is fixed by university finance policy and holds the audit trail that FR-001 and FR-005 depend on. Microservices lets the Ticketing Service be redeployed without re-testing or re-certifying the financial approval path. Fault isolation follows from the same split: if the Notification Service or the approval chain is down on event night, attendees are still admitted at the gate, whereas a single-tier deployment would take entry down with the portal.

Security Advantage. Every external request enters through one component, the API Gateway & Authentication Service, which authenticates the caller and issues a signed token carrying their role. NFR-002 — restricting budget approvals, ticket management and administrative records to authorised roles — is therefore enforced at a single auditable choke point instead of being re-implemented behind each screen. The domain services are not publicly routable, so the blast radius of any one compromised component is bounded by the interfaces it is allowed to call: a scanner device stolen from the venue gate holds credentials that reach only ITicketValidate and can never reach IBudgetApproval or the Proposal & Budget Store. Separating the ticket data from the budget data means the high-exposure check-in path never touches financial records at all.

Performance Benefit. NFR-001 requires QR validation in under 100 ms at peak. The Check-In Validation Service is stateless and answers from a read-optimised Ticket Store replica holding only ticket identifier, status and event — it carries none of the approval-workflow logic, reporting queries or session state of the main portal, so per-request work stays small and predictable. Because the service is stateless it scales horizontally: check-in traffic is absorbed by adding replicas rather than queueing behind proposal submissions and admin reports, which is exactly what would consume the 100 ms budget on a shared server.

Alternatives considered. Layered was rejected because its single deployment unit forces the bursty check-in path and the slow approval workflow to scale together. Client-Server was rejected because a central server is a single point of failure precisely at the moment it matters most — the gate, at event start.

Realised by: Portal Web Client, API Gateway & Authentication Service, Event Proposal Manager (= 'Order Manager'), Budget Approval Service (= 'Payment Service'), Ticketing Service, Check-In Validation Service, Notification Service, plus the Proposal & Budget Store and Ticket Store — connected by IPortalAPI, IProposalMgmt, IBudgetApproval, ITicketIssue, ITicketValidate, ITicketLookup and INotify. See Component_Diagram.pdf.